

Firewall, IDS, and IPS Implementation Report

Implementation Report: Firewall, IDS, and IPS Controls

Project: AI-Agentic SOC Home Lab **Platform:** Wazuh 4.14.7 (Manager / Indexer / Dashboard, Docker deployment) **Host:** macOS (UTM virtualization) — Ubuntu 24.04 monitored endpoint **Date:** September 10, 2026

1. Overview

This report documents the implementation of three complementary security controls within the lab's Wazuh-based SOC environment:

1. A **firewall rule** restricting network exposure of the Wazuh stack
2. An **IDS configuration** to detect SSH brute-force activity on a monitored endpoint
3. An **IPS configuration** using Wazuh Active Response to automatically block the offending IP

Together these demonstrate the detect → decide → respond pipeline: the firewall rule limits attack surface, the IDS rule detects malicious behavior, and the IPS action neutralizes it in near real time.

2. Firewall Rule Implementation

Objective: Prevent the Wazuh manager's API and agent-enrollment ports from being reachable outside the local network, since `docker-compose.yml` publishes ports on all interfaces by default.

Control: Host-level `ufw` (Uncomplicated Firewall) rule on the manager host, scoping exposed Wazuh ports to the LAN subnet only.

```
# Allow Wazuh agent + API ports only from the trusted LAN range
sudo ufw allow from 10.11.0.0/22 to any port 1514 proto tcp comment 'Wazuh agent events'
sudo ufw allow from 10.11.0.0/22 to any port 1515 proto tcp comment 'Wazuh agent enrollment'
sudo ufw allow from 10.11.0.0/22 to any port 55000 proto tcp comment 'Wazuh API'

# Deny the same ports from any other source
sudo ufw deny 1514,1515,55000/tcp

sudo ufw enable
sudo ufw status verbose
```

Result: Agent traffic and API calls are accepted only from `10.11.0.0/22` (the lab's local subnet); any external attempt to reach these ports is dropped at the host firewall before it reaches the Docker-published port.

3. IDS Configuration

Objective: Detect repeated failed SSH authentication attempts against a monitored Ubuntu endpoint (a common precursor to compromise).

Control: Wazuh's built-in SSHD decoder/ruleset plus a custom correlation rule in `local_rules.xml` that flags a burst of failures as a brute-force attempt.

`/var/ossec/etc/rules/local_rules.xml` :

```
<group name="local,syslog,sshd,">
  <rule id="100010" level="10" frequency="6" timeframe="120">
    <if_matched_sid>5716</if_matched_sid>
    <description>SSHD brute force: multiple failed logins from same source in 2
minutes</description>
    <mitre>
      <id>T1110</id>
    </mitre>
    <group>authentication_failures,</group>
  </rule>
</group>
```

Agent-side (`/var/ossec/etc/ossec.conf` on the monitored endpoint) ensures the auth log is collected:

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/auth.log</location>
</localfile>
```

Result: Six or more failed SSH logins from the same source within 120 seconds trigger rule `100010` at severity level 10, visible on the Wazuh dashboard under Security Events.

4. IPS Configuration

Objective: Automatically block the source IP once the brute-force rule fires, without waiting on an analyst.

Control: Wazuh Active Response bound to rule `100010` , executing the built-in `firewall-drop` script on the affected agent.

`/var/ossec/etc/ossec.conf` (manager):

```
<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100010</rules_id>
  <timeout>600</timeout>
</active-response>
```

Result: When rule `100010` fires, the agent inserts an `iptables -A INPUT -s <attacker_ip> -j DROP` (or equivalent `nft` / `ufw` rule depending on the script variant) on the targeted host, blocking the source for 10 minutes before automatic removal — a self-expiring, low-maintenance containment action suited to a lab environment.

5. Example Detected Event

The following is a representative Wazuh alert illustrating the pipeline in action (formatted from the dashboard’s alert view — actual field values will vary by run and should be captured from your own `alerts.json` for the final submission):

```
{
  "timestamp": "2026-09-10T14:22:07.481+0000",
  "rule": {
    "id": "100010",
    "level": 10,
    "description": "SSHD brute force: multiple failed logins from same source in 2 minutes",
    "mitre": { "id": ["T1110"] }
  },
  "agent": { "name": "ubuntu-endpoint-01", "ip": "10.11.3.42" },
  "data": {
    "srcip": "203.0.113.77",
    "srcport": "51422",
    "dstuser": "root"
  },
  "full_log": "sshd[2211]: Failed password for root from 203.0.113.77 port 51422 ssh2",
  "active_response": {
    "command": "firewall-drop",
    "status": "success",
    "action": "iptables DROP rule added for 203.0.113.77"
  }
}
```

Interpretation: Source `203.0.113.77` attempted repeated root logins against `ubuntu-endpoint-01`. The IDS rule correlated six failures within the timeframe and fired at level 10; Active Response immediately dropped the source IP at the host firewall, completing the detect-and-respond cycle without manual intervention.

6. Summary

Control	Layer	Mechanism	Trigger
Firewall rule	Network	<code>ufw</code> LAN-scoped allow/deny	Static policy
IDS	Detection	Wazuh correlation rule <code>100010</code>	6 failed SSH logins / 120s
IPS	Response	Active Response <code>firewall-drop</code>	Rule <code>100010</code> match

This layered configuration keeps the Wazuh management plane off the public interface, detects credential-based attacks against monitored hosts, and contains them automatically — a minimal but complete example of preventive, detective, and responsive controls working together.